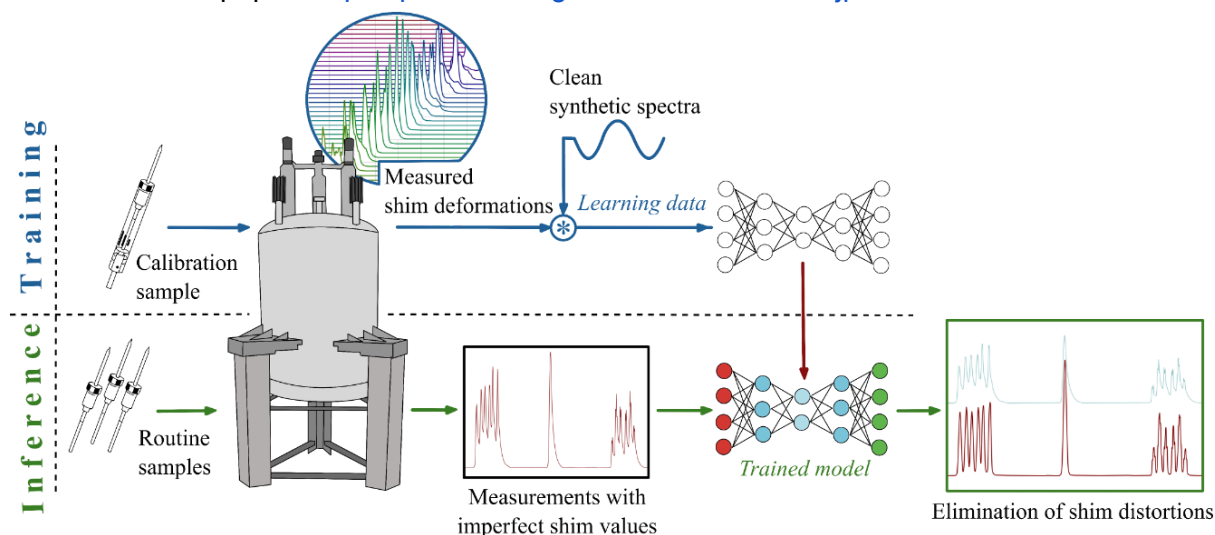


Correcting lineshape distortions using ShimNet

Krzysztof Kazimierczuk, Sylwia Jopa, Marek Bukowicki

ShimNet is a neural network for post-acquisition correction of NMR spectra distorted by magnetic field inhomogeneity. The network is trained using synthetic spectra convolved with “natural” distortions generated and measured in a calibration procedure. The software is described in the paper: <https://pubs.acs.org/doi/full/10.1021/acs.jpcb.5c02632>



The hands-on will cover:

1. The use of ShimNet program and a web service.
2. The collection of calibration spectra and the extraction of distortions
3. The generation of training spectra
4. The example neural network training
5. The comparison of networks obtained using different training parameters.

The ShimNet is a standalone Python script, generally run from the command line. However, for better understanding and consistent visualization of the results, we will work today in the Google Colab environment.

To start, log in to Google Colab and type:

```
from google.colab import drive
drive.mount('/content/drive')
%cd /content
!git clone -b workshops https://github.com/center4ml/shimnet.git
!mv shimnet /content/drive/MyDrive/shimnet_workshop
```

This will connect Colab to your Google Drive (login needed) and download the ShimNet package with example data from GitHub.

Open the notebook named `ShimNet_workshop1` from your Google Drive folder `shimnet_workshop/Workshop_Colab_Notebooks`.

Keep Google Drive open in a separate tab or window.

Run the first cell to enter the `shimnet_workshop` folder, install all the required Python packages, and download the data.

1. The use of ShimNet program and webservice.

The cell named *ShimNet Predict* contains the command starting the ShimNet reconstruction. The following arguments are provided to the program:

- input (distorted) spectrum (text file with two columns)
- model (trained network)
- config file
- output folder
- suffix for the output files (optional)

**Run the cell to correct the distorted spectrum using ShimNet (set proper input).
Run the next cell named `Plot results` (set the proper variable names).**

The input argument can be changed to run ShimNet on the other spectra (listed below). For some spectra, the model and config file have to be changed from 600 MHz to 700 MHz.

`"Asarone_deshimmed"`

Spectrum of 14 mM Asarone in MeOD (intentionally) deshimmed using Z1 and Z2 shims.

`"Asarone_reduce_volume"`

Sample shimmed, then removed from the magnet, 20 ul subtracted, sample inserted again and spectrum re-acquired without shimming.

`"lineshape_deshimmed"`

Spectrum of 1% CHCl_3 in acetone-d (intentionally) deshimmed using Z1 and Z2 shims.

`"multi_singlets_deshimmed"`

Spectrum of a mixture giving several singlets in a ^1H NMR spectrum.

`"styrene_1mm_600MHz"`

Spectrum of 3 M styrene in CDCl_3 . Field homogeneity was distorted by vertically shifting the sample by 1mm in the spinner.

`"2etnaft_600MHz_deshimmed"`

Spectrum of 3 M 2-ethylnaphthalene in CDCl_3 . Wrongly shimmed in Z1 and Z2.

Now, we will check the effect of the intensity range on the ShimNet performance.

Run the cell named “SimNet predict - high intensity range”. It performs the reconstruction on two variants of a spectrum of Cresol Red sample:

```
"Cresol_Red_after_different_sample"
```

Spectrum of 0.78 mM Cresol Red in (wet) DMSO with a huge H₂O peak present. Shims adjusted using a different sample. The range of signal intensities is very high, rarely present in the training data. Thus, the correction is suboptimal. Compare with the next spectrum.

```
"Cresol_Red_after_different_sample_cut"
```

Spectrum of 0.78 mM Cresol Red in (wet) DMSO with huge H₂O peak zeroed. Shims adjusted using a different sample.

Run the cell named “Plot results - high intensity range” to compare the results for the Cresol Red sample with and without H₂O peak.

Open the web browser and go to

<https://huggingface.co/spaces/NMR-CeNT-UW/ShimNet>

Upload the selected dataset and run the processing.

2. The collection of calibration spectra and the extraction of distortions

ShimNet requires the collection of calibration spectra acquired in inhomogeneous magnetic fields with various spatial profiles. These are generated by missetting the Z1 and Z2 shim coil currents in a controlled way. Then, the distortion functions are deconvolved from the calibration spectra and used to generate the training data for the neural network.

This notebook demonstrates:

- How the calibration data is collected
- The idea of convolution and deconvolution
- How shim coil response functions are extracted from the calibration data

Open the Colab notebook named ShimNet_workshop2

Run the cell named Environment setup to connect to Google Drive and install packages

Run the second cell to see the Python script for acquiring the calibration data on the Agilent machine.

The third cell demonstrates the concept of convolution. Run it to see how the simulated spectrum is distorted by a “triangular” function.

In the next cell we try to determine the distortion function from the distorted spectrum by fitting it with the convolution of the original spectrum and modelled distortion. Run it and check the plots.

The first plot shows how the distortion function is recovered. The second plot shows how the convolution of the recovered distortion and original spectrum matches the distorted spectrum. Change the fitting window (`response_function_estimate_length`) and check the effect.

In the next cell, we repeat the calculations for the noisy spectrum. Adjust the noise levels to observe the impact on the fit.

In the fourth cell, we use only a part of the spectrum to fit the distortion.

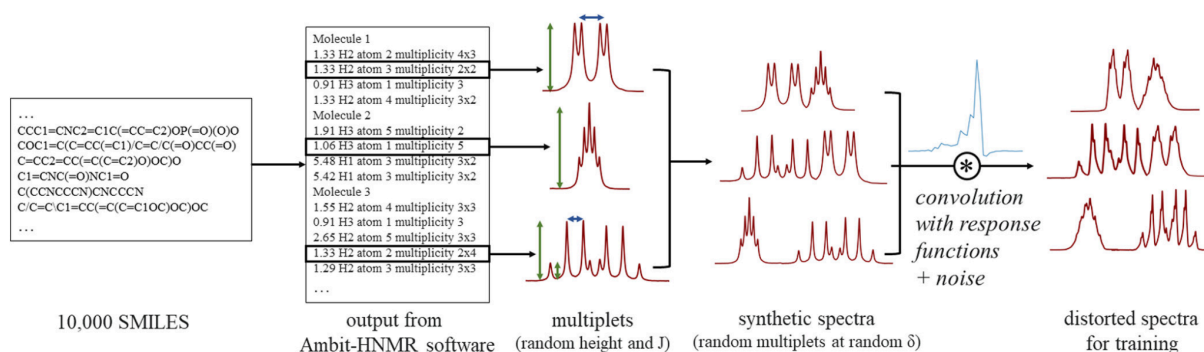
This is the procedure used in the Reference Deconvolution method - we fit only one, well-resolved peak whose “original” lineshape can be safely assumed.

In the last cell, we will extract the experimental shim coil response functions (SCRFs). Run it to see the plots of example CHCl_3 peaks and deconvolved distortion functions. Deconvolved functions are stored in a file and will be used in the next step to generate training spectra. The extraction script uses the following parameters:

```
# input
data_dir
    folder with raw FIDs from the calibration loop
opti_fid_path = "./opti_spectra/opti_total6.fid"
    folder with raw FID of optimally shimmed calibration sample

# output
spectra_file = "./sample_run/total.npy"
    file with parts of spectra used for deconvolution
spectra_file_names = "./sample_run/total.csv"
    file with names of fids from calibration
opi_spectrum_file = "./sample_run/opti.npy"
    file with part of opti spectrum used for calibration
responses_file = "./sample_run/scrif_shimotor9.pt"
    extracted distortion functions
losses_file = "./sample_run/losses_scrif_shimotor9.pt"
    file with loss values of respective distortion functions deconvolution
```

3. The generation of training spectra



ShimNet is trained using thousands or millions of simulated spectra generated using:

- experimental distortion functions extracted from the calibration spectra (above)
- multiplets patterns from the text file
 - the format of Ambit NMR software, see <https://zenodo.org/records/4629851>
 - generated from ENAMINE database with typical small molecules
<https://enamine.net/compound-collections/real-compounds/real-database>
- parameters from the `config.yaml` file

Normally, the generation of training data and actual network training are realized within one script. For demonstration purposes, we split them here.

Open the notebook named `ShimNet_workshop3`.

Run the first cell to mount Google Drive and install required packages.

Run the second cell to display the config file used to set up the parameters of the generated training spectra.

The parameters have the following meaning:

<code>pixels</code>	number of points in the spectrum
<code>frq_step</code>	frequency step size [Hz]
<code>relative_frequency_min and _max</code>	range of the spectrum where multiplets are located (relative)
<code>number_of_signals_min and _max</code>	number of multiplets in the spectrum
<code>thf_min and _max</code>	spectrum height component range
<code>relative_height_min and _max</code>	multiplet height component range
<code>spectrum_width_min and _max</code>	spectrum width component range [Hz]
<code>relative_width_min and _max</code>	multiplet width component range
<code>trf_min and max</code>	Gaussian fraction in multiplet shape
<code>multiplicity_j1_min and _max</code>	A-X couplings
<code>multiplicity_j2_min and _max</code>	A-Y couplings
<code>spectrum_noise_min and _max</code>	Gaussian noise added

Run the next cell to generate the batch of 10 spectra according to the config file.

Run the cell named Display spectra to view the results. Change the `spectrum_index` to view various spectra in the batch.

In the next cell, we will modify the data generation parameters (config) and check the results. We use only “data” part of the config file. Play with the parameters and check the results.

In the next cells you can use files with different multiplet patterns:

`data/singlets.txt`

`data/singlets_doublets_triplets.txt`

4. The example neural network training

Now, we will run the actual network training. For the sake of time, it will be very short training, insufficient to generate an optimal network.

Open the notebook named `ShimNet_workshop4`.

Run the first cell to mount Google Drive and install required packages.

Change environment to GPU (Runtime -> Change Runtime Type -> T4 GPU)

The second cell contains the full config file, whose ‘data’ section was used in the previous exercise. Run it to create the `training_1` folder with `config.yaml` file.

The third cell runs the neural network training. The results are stored in `./my_model/training_1` Run the cell and see the output and files appearing in `./my_model/training_1`

The printed output consists of 6 columns:

batch number, total loss, averaged total loss (100 batches), 1st term, 2nd term, 3rd term of the loss function.

The loss function:

$$\mathcal{L} = \frac{1}{K} (|S_{\text{trg}} - S_{\text{out}}|^2 + |R_{\text{trg}} - R_{\text{out}}|^2 + |S_{\text{inp}} - S_{\text{out}} \times R_{\text{out}}|^2)$$

S_{trg} - synthetic (ideal) spectra

S_{inp} - corresponding distorted spectra (+noise)

S_{out} - reconstructed spectra

R_{trg} - response function (distortion) from the calibration

R_{out} - reconstructed distortion function

K - the number of spectra in a batch

The files are:

`_spectrum_clean`
reconstructed spectrum (S_{out})
`_response`
output response function (R_{out})
`_spectrum_noise`
reconstruction of a distorted spectrum (convolution of a reconstructed clean spectrum with an output response function)
`_attention`
attention intensity plot

The fourth cell runs the reconstruction using the trained network. Use the spectrum `lineshape2_deshimmed`. Zoom in on a singlet and then on the multiplet. What do you notice?

Now, change:

`atom_groups_data_file:` `data/multiplets_10000_parsed.txt`

to:

`atom_groups_data_file:` `data/singlets.txt`

and repeat the training. Then, reconstruct the lines

Examine the resulting singlet and multiplet in `lineshape2_deshimmed`.

5. The comparison of networks obtained using different training parameters.

As we have seen in the previous point, training an effective network in several minutes is impossible. Thus, we will now utilize the pre-trained “large” models to evaluate their performance.

Open the notebook named `ShimNet_workshop5`.

Run the first cell to mount Google Drive and install required packages.

The second and third cells are known from the first exercise. Use them to reconstruct spectra and display them. You can use `-suffix` option to extend the output filename e.g. by adding the model name.

First, use a model trained only on singlets: `shimnet_singlets_600MHz.pt`

Check how it performs on azarone, styrene, and lineshape spectra.

In particular, note:

- shape of multiplets at 6.08 ppm and 6.58 ppm in azarone spectra
- shape of doublets at 5.74-5.77 ppm in styrene spectra

- shape of singlets in lineshape sample spectra

Now, let's try models trained using larger ranges of intensities in the training spectra:

`2_600_standardabs.pt` and `2_600_narrowabs.pt`

Use them to correct the spectrum named `Cresol_Red_after_different_sample.pt`

Check the quality of the resulting spectra. Do models trained on a high range perform better?

Explanation: peak intensities in the training data depend on the parameters randomly generated between assumed minimum and maximum values: h_s (`thf_min` and `thf_max` in config file) and h_m (`relative_height_min` and `relative_height_max` in the config file):

model	thf_{min} (h_s min)	thf_{max} (h_s max)	$relative_height_{min}$ (h_m min)	$relative_height_{max}$ (h_m max)	Approx. dynamic range (normalized) = h_{m_max}/h_{m_min}	Approx. dynamic range (not normalized) = $(h_{m_max} * h_{s_max}) / (h_{m_min} * h_{s_min})$
<code>shimnet_600MHz.pt</code>	0.5	2	0.5	4	8	32
<code>2_600_standardabs.pt</code>	0.5	2	0.125	16	128	512
<code>2_600_narrowabs.pt</code>	1	1	0.125	16	128	128

6. Homework

In the `INPUT` folder, you will find a misshimmed 600 MHz spectrum containing only triplets named `triplets_homework.csv`. The corresponding shimmed spectrum is in the `OPTI_INPUT_SPECTRA`. **Train the dedicated neural network to correct. Generate training data by modifying a notebook `ShimNet_workshop3` and using a file containing only triplets (`data/triplets_homework.txt`).**

Train the dedicated neural network by modifying the notebook `ShimNet_workshop4` and reconstruct the well-shimmed spectrum.

Note 1: if you deleted the course folder, you may need to rerun `ShimNet_workshop1` and `ShimNet_workshop2` to download the calibration data and extract shim coil response functions again.

Note 2: Of course, with a limited number of distortion functions and fast training, the result will be poor. You can switch to a "large" model `./weights/shimnet_600MHz.pt` to see the difference.

Send the modified notebooks to:

k.kazimierzczuk@cent.uw.edu.pl